LAPORAN PRAKTIKUM

Modul 7: CRUD dan Upload File pada Laravel



Disusun Oleh:

Nama : Davin Aileenskie Sofiandy

NIM : 707012500135

Kelas : D4SIKC-49-04

Mata Kuliah : Pengembangan Aplikasi Berbasis Web

TELKOM UNIVERSITY

FAKULTAS ILMU TERAPAN

JURUSAN S1 TERAPAN SISTEM INFORMASI KOTA CERDAS

TAHUN AJARAN 2026

1. Tujuan Praktikum

Praktikum Modul 7 ini saya kerjakan untuk menambahkan fitur CRUD dan unggah berkas pada project Laravel. Project yang saya pakai bernama PRK_7 dan sudah memakai Laravel Breeze untuk autentikasi. Pada modul sebelumnya aplikasi baru bisa membedakan pengguna yang sudah login dan belum. Di modul ini saya menambahkan tabel uploads, model Upload, controller, route, dan empat view supaya setiap pengguna bisa mengelola berkas miliknya sendiri. Ada beberapa tujuan yang ingin saya capai.

1. Memahami konsep CRUD, resource controller, dan resource route pada Laravel.
2. Memahami konsep unggah berkas, form multipart, validasi file, disk public, dan symbolic link storage.
3. Membuat model Upload beserta migration tabel uploads yang terhubung ke tabel users.
4. Membuat UploadController dengan tujuh method CRUD, yaitu index, create, store, show, edit, update, dan destroy.
5. Mendaftarkan Route::resource('uploads') di dalam middleware auth dan menambahkan menu Berkas pada navigasi Breeze.
6. Membuat view index, create, edit, dan show untuk menampilkan, menambah, mengubah, dan menghapus berkas.
7. Menguji unggah gambar dan PDF, validasi jenis dan ukuran file, pagination, perubahan judul, penghapusan berkas, dan pembatasan akses pemilik.

2. Alat dan Bahan

Praktikum ini saya kerjakan pada laptop pribadi dengan sistem operasi Windows 11 Pro. Perangkat lunak yang dipakai sebagian besar sudah tersedia dari praktikum sebelumnya. Laravel Breeze dan Node.js juga sudah terpasang karena project PRK_7 memakai tampilan Breeze.

2.1 Perangkat Lunak

Perangkat Lunak	Versi / Keterangan
XAMPP (Apache & MySQL)	Terpasang pada D:\xampp
MySQL / MariaDB	10.4.32, server database melalui XAMPP, database db_crud_breeze
phpMyAdmin	5.2.1, untuk memeriksa tabel uploads
PHP	8.2.12 (ZTS Visual C++ 2019 x64)
Composer	2.10.3
Laravel Framework	12.70.1
Laravel Breeze	2.4.2
Node.js	25.6.0
NPM	11.8.0
Vite	7.3.7
Visual Studio Code	Editor kode dan terminal project
Git	2.53.0.windows.1
Google Chrome	Untuk menjalankan dan menguji aplikasi

3. Langkah-Langkah Praktikum

Project PRK_7 memakai Laravel 12.70.1 dan Laravel Breeze 2.4.2. Bagian autentikasi sudah ada dari modul sebelumnya, jadi langkah pada modul ini saya mulai dari pembuatan model Upload sampai pengujian CRUD. Saya mengikuti jalur Breeze pada subbab 7.4.4. Jalur manual pada subbab 7.4.5 tidak saya campur ke project ini karena route dan layoutnya berbeda.

3.1 Menyiapkan Database

Database yang dipakai bernama db_crud_breeze. Saya membuat database itu melalui phpMyAdmin, lalu menyesuaikan bagian database pada file .env. Nilai DB_CONNECTION diisi mysql, DB_HOST 127.0.0.1, DB_PORT 3306, DB_DATABASE db_crud_breeze, dan DB_USERNAME root. DB_PASSWORD dibiarkan kosong mengikuti konfigurasi XAMPP bawaan yang tidak memberi password pada user root. Setelah itu php artisan migrate dijalankan untuk membuat tabel bawaan Laravel dan Breeze. Tabel yang terbentuk antara lain users, sessions, cache, jobs, dan migrations. Tabel uploads belum ada karena migration-nya dibuat pada langkah berikutnya.

3.2 Membuat Model dan Migration Upload

Model dan migration dibuat sekaligus dengan perintah php artisan make:model Upload -m. Perintah itu menghasilkan app/Models/upload.php dan file migration 2026_10_26_012213_create_uploads_table.php. Nama tabel otomatis menjadi uploads. Saya mengubah method up() agar tabel menyimpan pemilik berkas, judul, keterangan, nama asli file, dan path file.

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('uploads', function (Blueprint $table) {
            $table->id();
            $table->foreignId('user_id')->constrained()->cascadeOnDelete();
            $table->String('judul');
            $table->text('keterangan')->nullable();
            $table->string('filename');
            $table->string('filepath');
            $table->timestamps();
        });
    }
    public function down(): void
    {
        Schema::dropIfExists('uploads');
    }
};
```

Kolom user_id memakai foreignId()->constrained()->cascadeOnDelete(). Relasi itu menghubungkan setiap berkas ke tabel users. Kalau akun pemilik dihapus, data berkasnya ikut terhapus. Kolom keterangan diberi nullable() karena keterangan boleh kosong. Kolom filename menyimpan nama asli dari komputer pengguna, sedangkan filepath menyimpan alamat file di storage. Isi file tidak masuk ke database. Setelah migration dijalankan, tabel uploads muncul di phpMyAdmin.

3.3 Model Upload

Model Upload dipakai untuk mengambil dan menyimpan data. Saya mengisi $table dengan 'uploads', lalu mendaftarkan lima kolom di $fillable supaya bisa diisi lewat Upload::create() dan update(). Relasi user() memakai belongsTo(User::class), sehingga nama pemilik bisa dipanggil dengan $upload->user->name.

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class upload extends Model
{
    protected $table = 'uploads';
    protected $fillable = [
        'user_id',
        'judul',
        'keterangan',
        'filename',
        'filepath',
    ];

    public function user()
    {
        return $this->belongsTo(User::class);
    }
}
```

Nama kelas pada file ini tertulis upload dengan huruf u kecil. PHP tetap mencocokkan nama kelas tanpa membedakan huruf besar dan kecil, dan pemanggilan Upload::create() di controller berjalan. Meskipun begitu, penulisan yang mengikuti nama file dan PSR-4 seharusnya memakai

Upload dengan huruf besar. Saya biarkan karena aplikasi sudah berjalan dan tidak ingin mengubah nama kelas di tengah pengujian.

3.4 UploadController

Controller dibuat dengan php artisan make:controller UploadController --resource. Ada tujuh method yang dipakai, yaitu index, create, store, show, edit, update, dan destroy. Method index mengambil data dengan Upload::where('user_id', Auth::id())->latest()->paginate(5). Filter itu membuat setiap pengguna hanya melihat berkas miliknya, urut dari yang terbaru, lima data per halaman.

```php
public function store(Request $request)
{
    $request->validate([
        'judul' => 'required|string|max:255',
        'keterangan' => 'nullable|string',
        'file' => 'required|file|mimes:jpg,jpeg,png,pdf,doc,docx|max:2048',
    ]);

    $file = $request->file('file');
    $path = $file->store('uploads', 'public');

    Upload::create([
        'user_id' => Auth::id(),
        'judul' => $request->judul,
        'keterangan' => $request->keterangan,
        'filename' => $file->getClientOriginalName(),
        'filepath' => $path,
    ]);

    return redirect()->route('uploads.index')->with(
        'success',
        'Berkas berhasil diunggah.'
    );
}
```

Method store memvalidasi judul, keterangan, dan file. Aturan file memakai required, file, mimes:jpg,jpeg,png,pdf,doc,docx, dan max:2048. File kemudian disimpan dengan $file->store('uploads', 'public'). Path hasil store() masuk ke kolom filepath, sedangkan nama asli file masuk ke filename. Setelah data tersimpan, pengguna diarahkan ke halaman index dengan pesan Berkas berhasil diunggah.

```php
public function update(Request $request, Upload $upload)
{
    abort_if($upload->user_id !== Auth::id(), 403);

    $request->validate([
        'judul' => 'required|string|max:255',
        'keterangan' => 'nullable|string',
        'file' => 'nullable|file|mimes:jpg,jpeg,png,pdf,doc,docx|max:2048',
    ]);

    $data = $request->only('judul', 'keterangan');

    if ($request->hasFile('file')) {
        Storage::disk('public')->delete($upload->filepath);

        $file = $request->file('file');
        $data['filename'] = $file->getClientOriginalName();
        $data['filepath'] = $file->store('uploads', 'public');
    }

    $upload->update($data);

    return redirect()->route('uploads.index')->with(
        'success',
        'Berkas berhasil diperbarui.'
    );
}
```

Method update memakai aturan file yang sama, tetapi file bersifat nullable. Kalau pengguna memilih file baru, file lama dihapus dengan Storage::disk('public')->delete($upload->filepath), lalu file baru disimpan. Kalau input file kosong, judul dan keterangan tetap diperbarui dan file lama dipakai. Method show, edit, update, dan destroy memanggil abort_if($upload->user_id !== Auth::id(), 403). Pemeriksaan itu menolak pengguna lain yang mencoba membuka atau mengubah berkas bukan miliknya.

```php
public function destroy(Upload $upload)
{
    abort_if($upload->user_id !== Auth::id(), 403);

    Storage::disk('public')->delete($upload->filepath);
    $upload->delete();

    return redirect()->route('uploads.index')->with(
        'success',
        'Berkas berhasil dihapus.'
    );
}
```

Method destroy menghapus file fisik dari storage lebih dulu, lalu menghapus baris di tabel uploads. Urutan itu penting supaya path file masih tersedia saat file dihapus. Setelah itu pengguna kembali ke halaman index dengan pesan Berkas berhasil dihapus.

3.5 Route Resource dan Menu Berkas

Route resource didaftarkan di routes/web.php di dalam grup middleware auth. Satu baris Route::resource('uploads', UploadController::class) membuat tujuh route sekaligus. Karena berada di dalam grup auth, pengguna yang belum login diarahkan ke halaman login ketika membuka /uploads.

```php
<?php

use App\Http\Controllers\ProfileController;

use App\Http\Controllers\UploadController;
use Illuminate\Support\Facades\Route;

Route::get('/', function () {
    return view('welcome');
});
Route::get('/dashboard', function () {
    return view('dashboard');
})->middleware(['auth', 'verified'])->name('dashboard');
Route::middleware('auth')->group(function () {
    Route::get('/profile', [ProfileController::class, 'edit'])->name('profile.edit');

    Route::patch('/profile', [ProfileController::class, 'update'])-
>name('profile.update');

    Route::delete('/profile', [ProfileController::class, 'destroy'])-
>name('profile.destroy');

    // CRUD Berkas: hanya dapat diakses setelah login
    Route::resource('uploads', UploadController::class);
});
require __DIR__ . '/auth.php';
```

Hasil php artisan route:list --name=uploads menunjukkan tujuh route: uploads.index, uploads.store, uploads.create, uploads.show, uploads.update, uploads.destroy, dan uploads.edit. Route update memakai method PUT/PATCH, sedangkan route destroy memakai DELETE. Form HTML hanya mendukung GET dan POST, jadi kedua method itu dikirim lewat @method('PUT') dan @method('DELETE').

Menu Berkas saya tambahkan di resources/views/layouts/navigation.blade.php tepat di bawah menu Dashboard. Komponen x-nav-link memakai route uploads.index dan :active request()->routeIs('uploads.*') supaya menu terlihat aktif saat halaman uploads dibuka.

```
<x-nav-link :href="route('uploads.index')"
    :active="request()->routeIs('uploads.*')">
    {{ __('Berkas') }}
</x-nav-link>
```

Menu Berkas pada versi ini hanya muncul di navigasi desktop. Tautan untuk menu mobile belum saya tambahkan di bagian Responsive Navigation Menu. Pada layar kecil, menu Dashboard tetap ada, tetapi menu Berkas tidak muncul. Saya catat bagian ini sebagai kekurangan karena modul meminta keduanya.

3.6 View Index

View index menampilkan daftar berkas dalam tabel. Kolomnya adalah No, Judul, Nama File, Tanggal Unggah, dan Aksi. Nomor urut memakai $uploads->firstItem() + $loop->index supaya tetap berlanjut di halaman kedua. Nama file dibuat sebagai tautan ke asset('storage/' . $upload->filepath) dan dibuka di tab baru. Tombol Lihat, Edit, dan Hapus berada di kolom Aksi. Tombol Hapus memakai form POST dengan @csrf dan @method('DELETE'), serta konfirmasi onclick sebelum data dihapus.

```blade
<table class="w-full border text-left">
    <thead class="bg-gray-100">
        <tr>
            <th class="border p-2">No</th>
            <th class="border p-2">Judul</th>
            <th class="border p-2">Nama File</th>
            <th class="border p-2">Tanggal Unggah</th>
            <th class="border p-2">Aksi</th>
        </tr>
    </thead>
    <tbody>
        @forelse ($uploads as $upload)
            <tr>
                <td class="border p-2">
                    {{ $uploads->firstItem() + $loop->index }}
                </td>
                <td class="border p-2">{{ $upload->judul }}</td>
                <td class="border p-2">
                    <a href="{{ asset('storage/' . $upload->filepath) }}"
                        target="_blank">
                        {{ $upload->filename }}
                    </a>
                </td>
                <td class="border p-2">
                    {{ $upload->created_at->format('d-m-Y H:i') }}
                </td>
                <td class="border p-2">
                    <a href="{{ route('uploads.show', $upload) }}">Lihat</a> |
                    <a href="{{ route('uploads.edit', $upload) }}">Edit</a> |
                    <form action="{{ route('uploads.destroy', $upload) }}"
                        method="POST" class="inline">
                        @csrf
                        @method('DELETE')
                        <button type="submit">Hapus</button>
                    </form>
                </td>
            </tr>
        @empty
            <tr>
                <td colspan="5" class="border p-2 text-center">
                    Belum ada berkas yang diunggah.
                </td>
            </tr>
        @endforelse
    </tbody>
</table>
```

Blok @forelse menampilkan baris data satu per satu. Kalau data kosong, blok @empty menampilkan pesan Belum ada berkas yang diunggah. Di bawah tabel ada $uploads->links() untuk pagination. Pada view ini tautan nama file memakai kelas text-blue-600underline tanpa spasi. Akibatnya warna biru dan garis bawah tidak muncul, tetapi tautannya tetap bisa diklik. Saya baru menyadari hal itu saat memeriksa hasil tampilan.

3.7 View Create

Form unggah berada di resources/views/uploads/create.blade.php. Form memakai method POST, route uploads.store, dan enctype multipart/form-data. Atribut enctype wajib ada supaya file ikut terkirim. Di dalam form ada input judul, textarea keterangan, input file, tombol Simpan, dan tautan Batal. Setiap input memakai komponen Breeze x-input-label, x-text-input, dan x-input-error. Nilai judul dan keterangan diisi kembali dengan old() ketika validasi gagal.

```
<form action="{{ route('uploads.store') }}" method="POST"
    enctype="multipart/form-data">
    @csrf

    <div class="mb-4">
        <x-input-label for="judul" value="Judul" />
        <x-text-input id="judul" name="judul" type="text"
            class="mt-1 block w-full" :value="old('judul')" required autofocus />
        <x-input-error :messages="$errors->get('judul')" class="mt-2" />
    </div>

    <div class="mb-4">
        <x-input-label for="keterangan" value="Keterangan (opsional)" />
        <textarea id="keterangan" name="keterangan" rows="3"
            class="mt-1 block w-full border-gray-300 rounded-md shadow-sm">{{
old('keterangan') }}</textarea>
        <x-input-error :messages="$errors->get('keterangan')" class="mt-2" />
    </div>

    <div class="mb-4">
        <x-input-label for="file"
            value="Pilih Berkas (jpg, jpeg, png, pdf, doc, docx - maks. 2 MB)" />
        <input id="file" name="file" type="file" class="mt-1 block w-full" required>
        <x-input-error :messages="$errors->get('file')" class="mt-2" />
    </div>

    <div class="flex items-center gap-4">
        <x-primary-button>Simpan</x-primary-button>
        <a href="{{ route('uploads.index') }}"
            class="text-gray-600 hover:text-gray-900">Batal</a>
    </div>
</form>
```

Input file memakai name='file' karena controller membaca $request->file('file'). Label input menyebut format jpg, jpeg, png, pdf, doc, dan docx dengan ukuran maksimal 2 MB. Atribut accept tidak dipakai, jadi pengguna bisa memilih file apa pun dari dialog. Pemeriksaan jenis file tetap dilakukan oleh validasi di controller.

3.8 View Edit

Form edit memakai route uploads.update dan @method('PUT'). Nilai awal judul dan keterangan diambil dari $upload dengan old('judul', $upload->judul) dan old('keterangan', $upload->keterangan). Di atas input file ada tautan File saat ini yang mengarah ke asset('storage/' .

$upload->filepath). Input file tidak memakai required, jadi pengguna boleh hanya mengubah judul atau keterangan.

```blade
<form action="{{ route('uploads.update', $upload) }}" method="POST"
    enctype="multipart/form-data">
    @csrf
    @method('PUT')

    <div class="mb-4">
        <x-input-label for="judul" value="Judul" />
        <x-text-input id="judul" name="judul" type="text"
            class="mt-1 block w-full"
            :value="old('judul', $upload->judul)" required />
        <x-input-error :messages="$errors->get('judul')" class="mt-2" />
    </div>

    <div class="mb-4">
        <x-input-label for="keterangan" value="Keterangan (opsional)" />
        <textarea id="keterangan" name="keterangan" rows="3"
            class="mt-1 block w-full border-gray-300 rounded-md shadow-sm">{{
old('keterangan', $upload->keterangan) }}</textarea>
        <x-input-error :messages="$errors->get('keterangan')" class="mt-2" />
    </div>

    <div class="mb-4">
        <p class="text-sm text-gray-600">File saat ini:
            <a href="{{ asset('storage/' . $upload->filepath) }}"
                target="_blank" class="text-blue-600 underline">
                {{ $upload->filename }}
            </a>
        </p>
        <x-input-label for="file"
            value="Ganti Berkas (kosongkan jika tidak ingin mengganti)"
            class="mt-2" />
        <input id="file" name="file" type="file" class="mt-1 block w-full">
        <x-input-error :messages="$errors->get('file')" class="mt-2" />
    </div>

    <div class="flex items-center gap-4">
        <x-primary-button>Update</x-primary-button>
        <a href="{{ route('uploads.index') }}"
            class="text-gray-600 hover:text-gray-900">Batal</a>
    </div>
</form>
```

Kalau pengguna memilih file baru, controller menghapus file lama dan menyimpan file baru. Kalau input file dibiarkan kosong, file lama tetap dipakai. Teks pada label menjelaskan hal itu supaya pengguna tidak mengira file lama ikut terhapus.

3.9 View Show

Halaman detail menampilkan judul, keterangan, nama file, nama pemilik, dan tanggal unggah. Nama pemilik diambil dari relasi $upload->user->name. Ekstensi file diambil dengan pathinfo($upload->filepath, PATHINFO_EXTENSION), lalu dicocokkan dengan daftar jpg, jpeg,

dan png. Kalau file berupa gambar, halaman menampilkan pratinjau. Kalau file berupa PDF atau Word, halaman menampilkan tombol Buka / Unduh Berkas.

```blade
<p><strong>Judul:</strong> {{ $upload->judul }}</p>
<p><strong>Keterangan:</strong> {{ $upload->keterangan ?? '-' }}</p>
<p><strong>Nama File:</strong> {{ $upload->filename }}</p>
<p><strong>Diunggah oleh:</strong> {{ $upload->user->name }}</p>
<p><strong>Tanggal Unggah:</strong>
    {{ $upload->created_at->format('d-m-Y H:i') }}</p>

@php
    $ext = strtolower(pathinfo($upload->filepath, PATHINFO_EXTENSION));
@endphp

@if (in_array($ext, ['jpg', 'jpeg', 'png']))
    <div>
        <img src="{{ asset('storage/' . $upload->filepath) }}"
            alt="{{ $upload->judul }}" class="max-w-sm rounded border">
    </div>
@else
    <div>
        <a href="{{ asset('storage/' . $upload->filepath) }}"
            target="_blank"
            class="inline-block px-4 py-2 bg-gray-800 text-white rounded">
            Buka / Unduh Berkas
        </a>
    </div>
@endif
```

Tombol dan pratinjau memakai URL asset('storage/' . $upload->filepath). URL itu hanya bisa dibuka kalau symbolic link public/storage sudah ada. Pada project ini folder public/storage belum tersedia saat pertama kali saya periksa, jadi saya menjalankan php artisan storage:link. Perintah itu membuat junction public/storage yang mengarah ke storage/app/public.

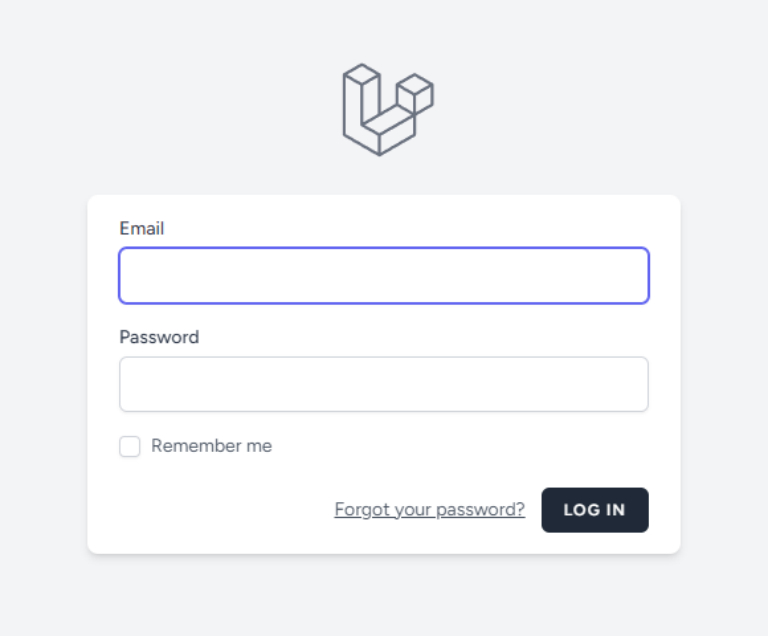
3.10 Menjalankan Aplikasi dan Menguji

Setelah semua bagian selesai, saya menjalankan php artisan serve pada alamat http://127.0.0.1:8000. Aset Breeze sudah tersedia di public/build, jadi saya tidak menjalankan npm run dev selama pengujian. Server saya biarkan terbuka, lalu pengujian dilakukan di Chrome. Urutan pengujiannya adalah membuka /uploads tanpa login, login memakai akun davin, membuka menu Berkas, mengunggah beberapa berkas, mencoba file yang tidak valid, membuka detail, mengubah judul, menghapus satu berkas, dan mencoba membuka berkas milik pengguna lain.

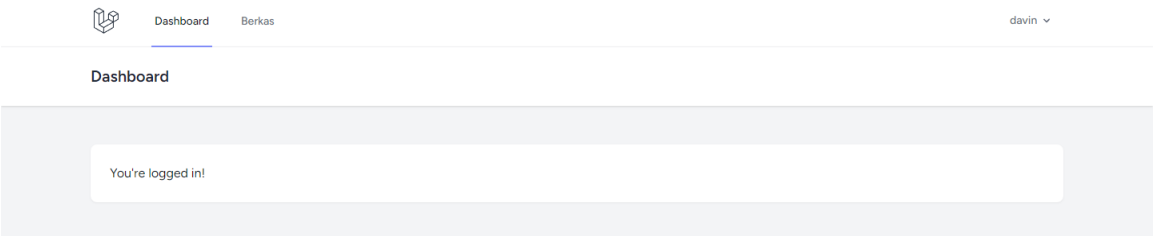
Untuk pengujian kepemilikan data, saya membuat satu akun kedua bernama Pengguna Lain. Akun itu dipakai untuk membuka /uploads/1, yaitu berkas milik akun davin. Hasilnya halaman 403 Forbidden. Daftar berkas akun kedua juga kosong karena query memakai where('user_id', Auth::id()).

4. Hasil dan Dokumentasi Program

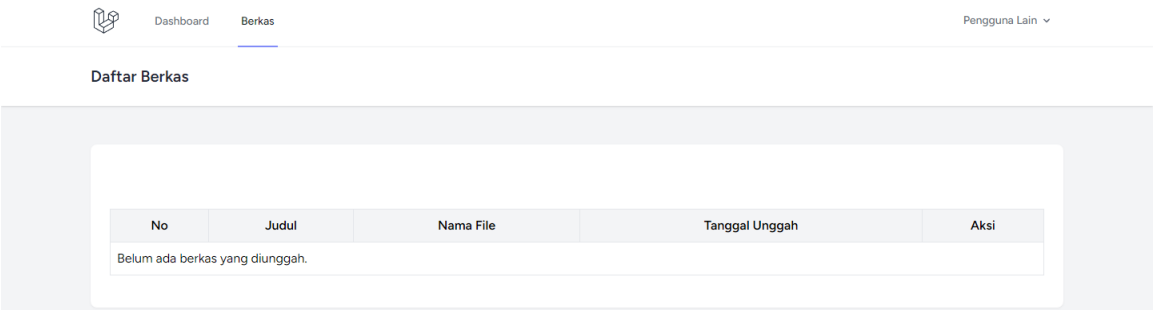
Seluruh alur CRUD berjalan. Saya berhasil mengunggah gambar dan PDF, menampilkan daftar dengan pagination, membuka detail, mengubah judul, dan menghapus berkas. Validasi menolak file ZIP dan file berukuran lebih dari 2 MB. Validasi juga menolak file DOCX, meskipun ekstensi itu tercantum pada aturan mimes. Bagian itu saya jelaskan lebih lanjut di analisis. Berikut dokumentasi hasil pengujian.



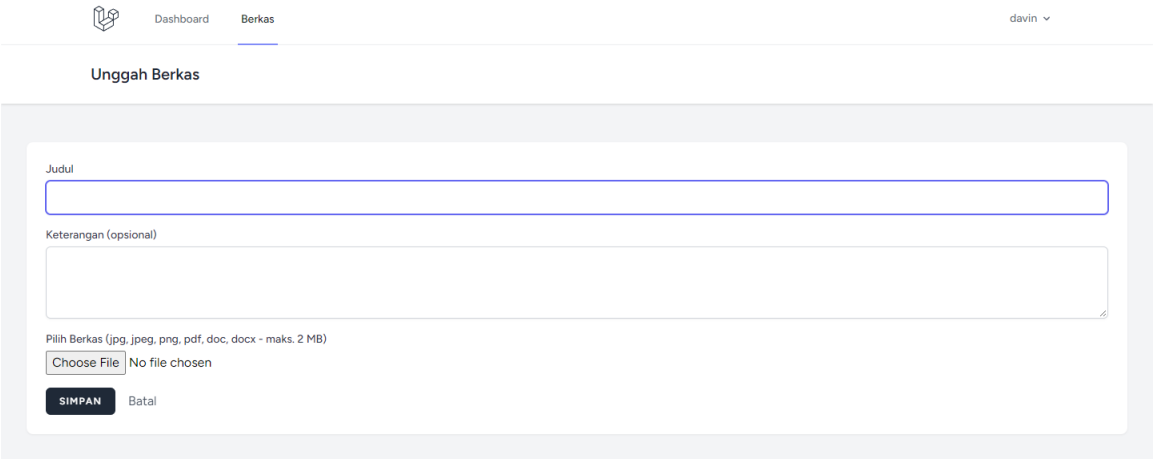
Gambar 1 menampilkan halaman login yang muncul ketika /uploads dibuka tanpa sesi. Alamat browser berubah dari /uploads menjadi /login karena middleware auth menahan permintaan.



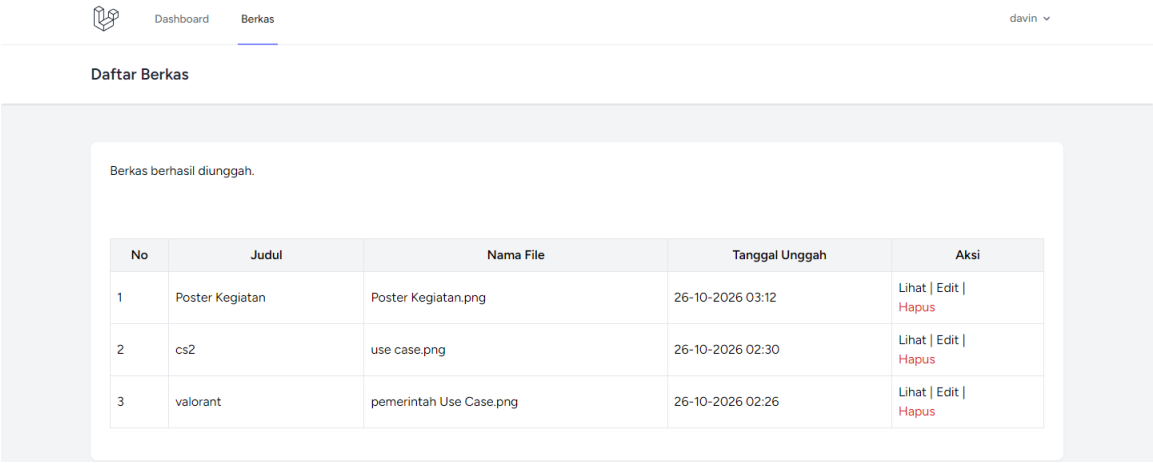
Gambar 2 menampilkan dashboard setelah login. Menu Berkas muncul di samping Dashboard, sedangkan nama davin terlihat di kanan atas.



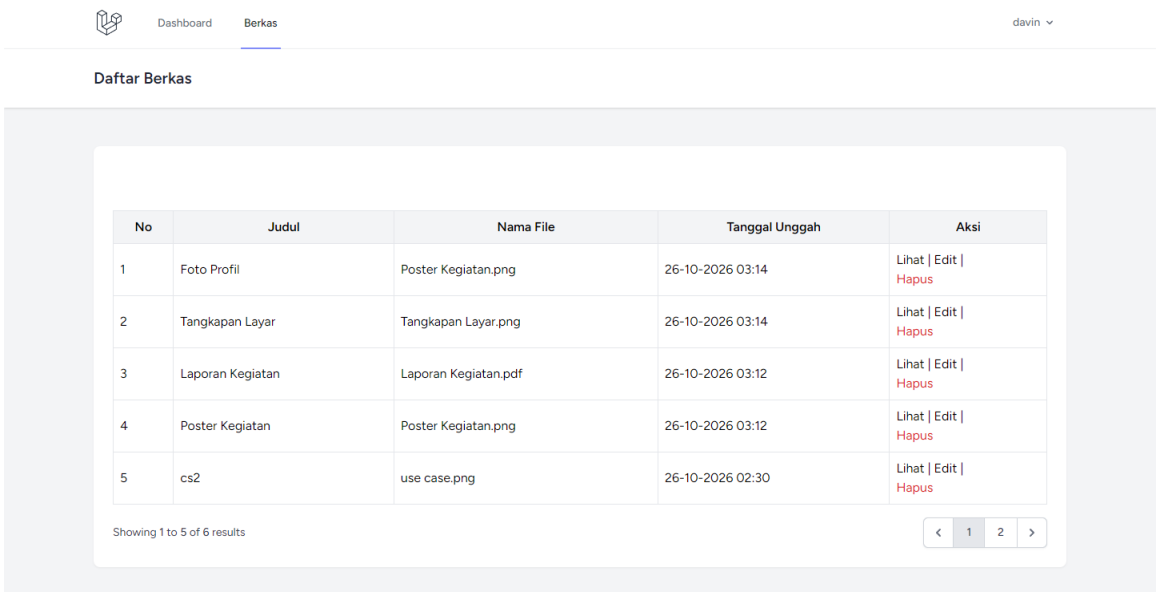
Gambar 3 menampilkan daftar berkas akun Pengguna Lain yang masih kosong. Pesan Belum ada berkas yang diunggah berasal dari blok @empty.



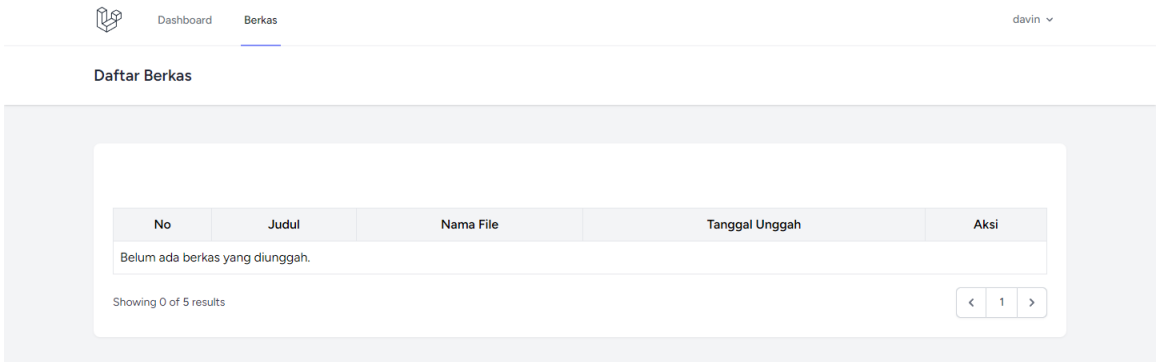
Gambar 4 menampilkan form unggah dengan input Judul, Keterangan, dan Pilih Berkas. Form memakai enctype multipart/form-data dan mengirim data ke route uploads.store.



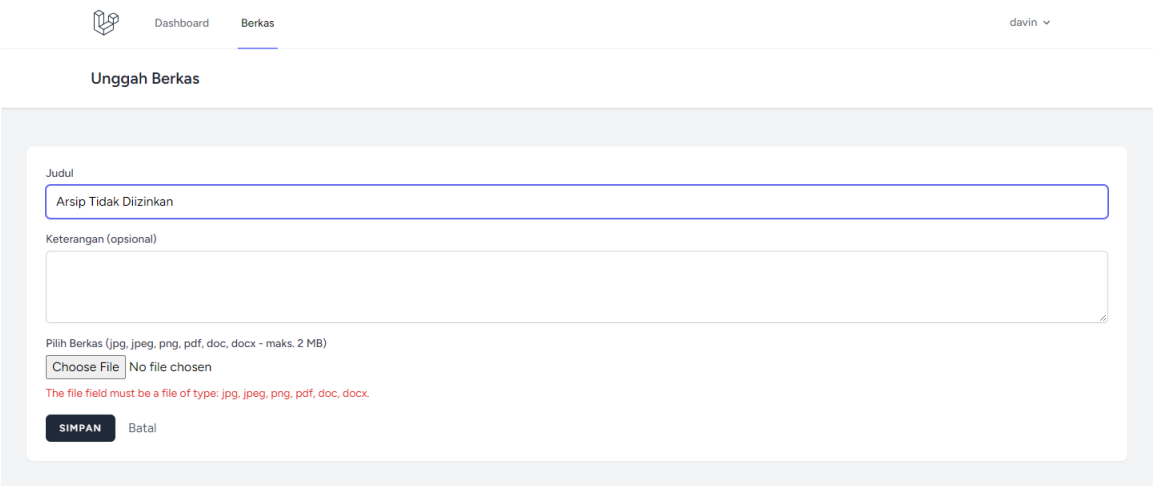
Gambar 5 menampilkan pesan Berkas berhasil diunggah setelah tombol Simpan ditekan. Berkas Poster Kegiatan langsung muncul di baris pertama.



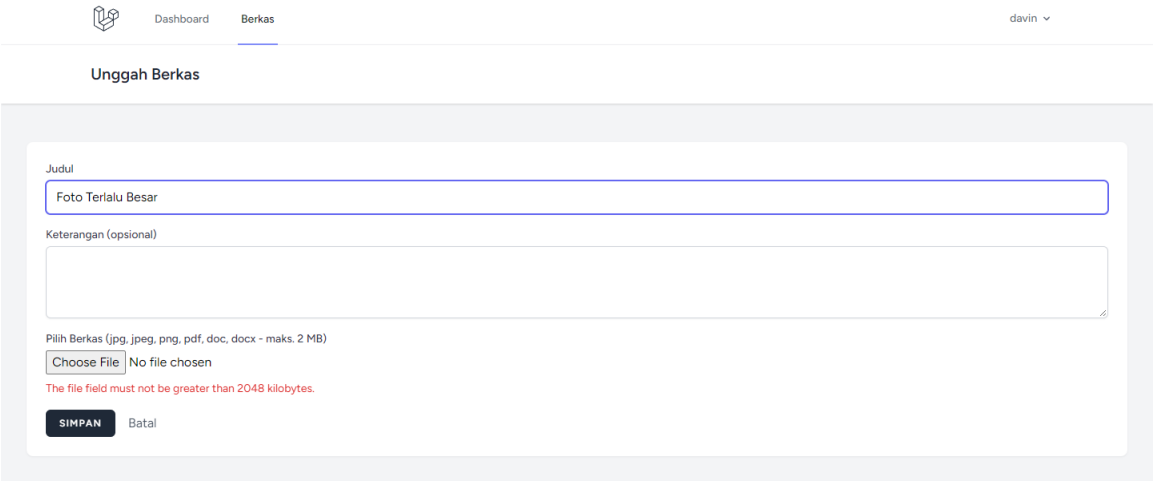
Gambar 6 menampilkan daftar berkas halaman pertama. Ada lima baris, tombol pagination di kanan bawah, dan nomor urut 1 sampai 5.



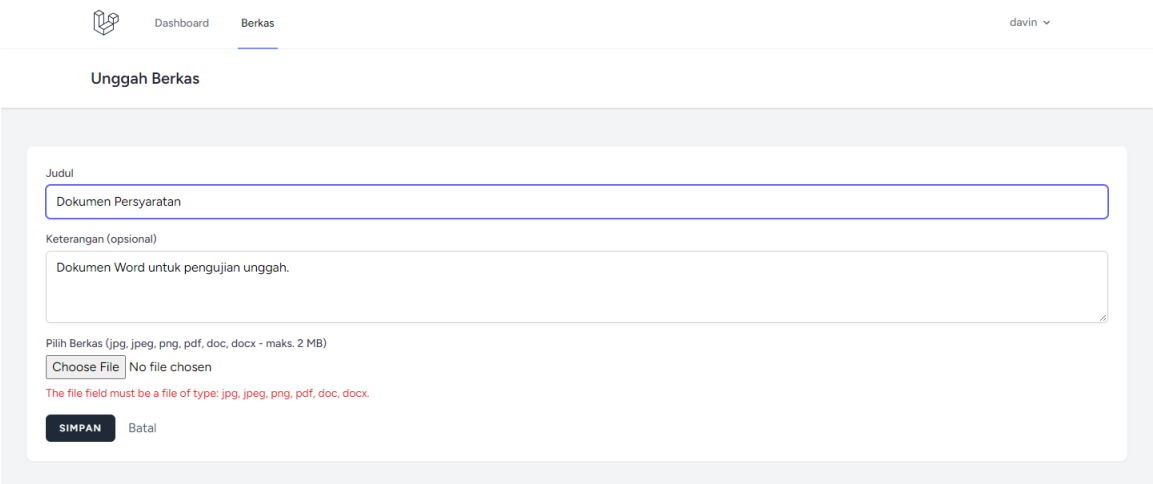
Gambar 7 menampilkan halaman kedua. Hanya ada satu baris dengan nomor 6, jadi nomor urut tetap berlanjut dari halaman sebelumnya.



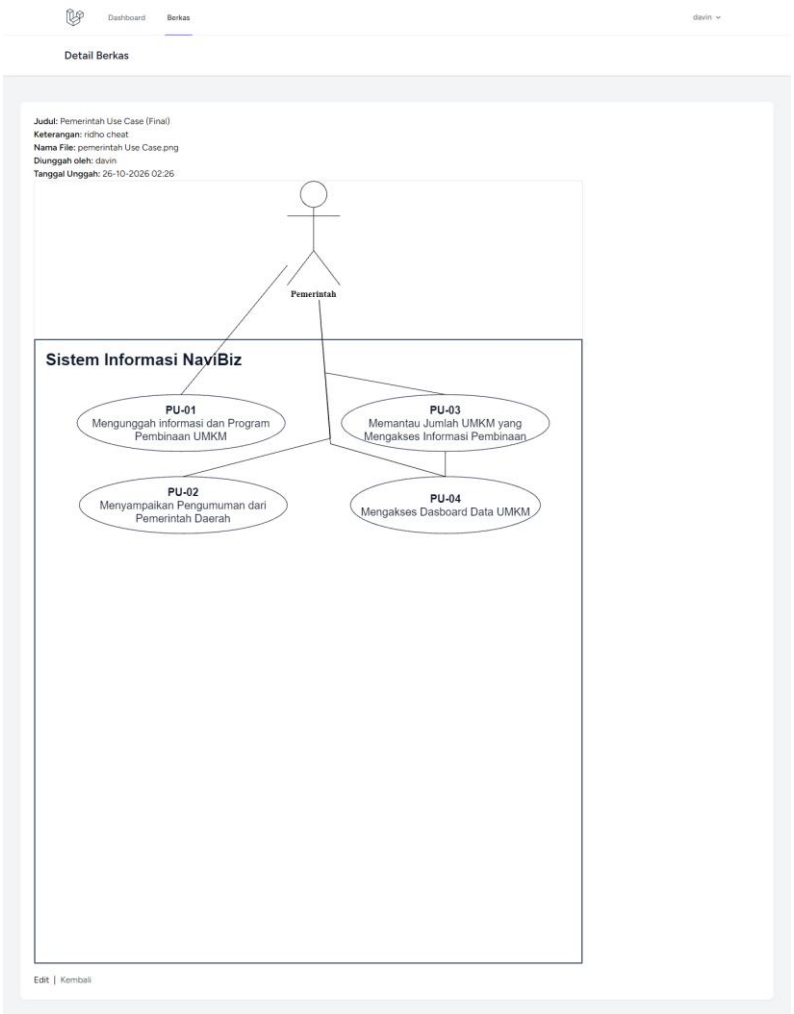
Gambar 8 menampilkan pesan galat ketika file ZIP diunggah. Validasi mimes menolak file dan input file dikosongkan lagi.



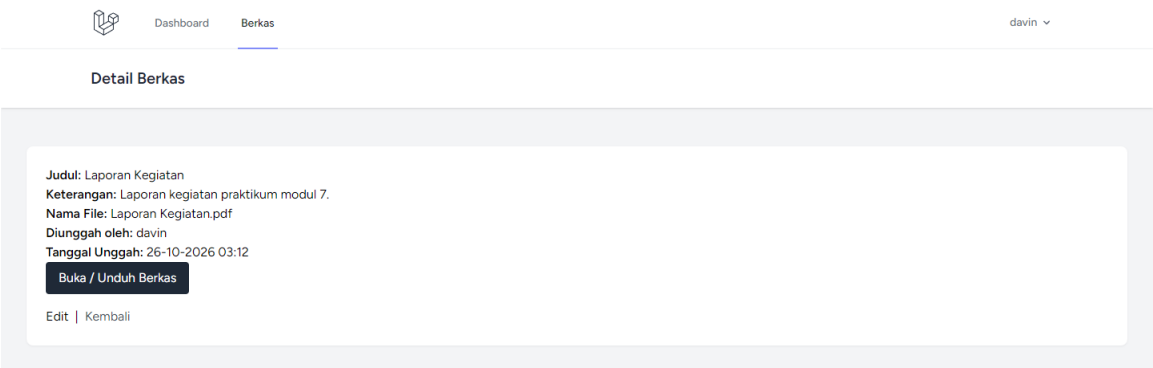
Gambar 9 menampilkan pesan galat ketika file PNG berukuran sekitar 6,4 MB diunggah. Aturan max:2048 menolak file yang melebihi 2 MB.



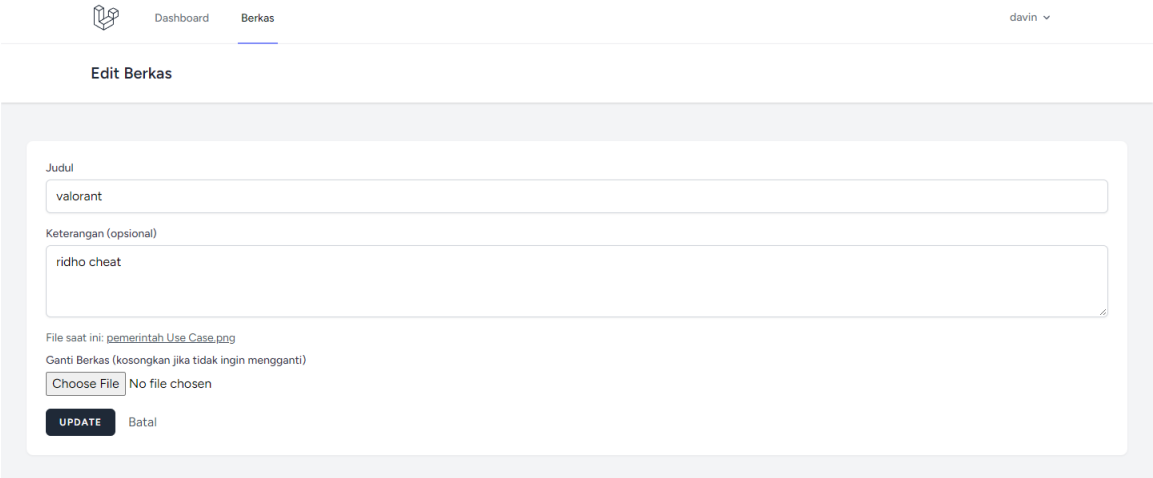
Gambar 10 menampilkan pesan galat ketika file DOCX diunggah. Pesan yang muncul sama dengan pesan untuk ZIP, padahal docx sudah tercantum pada aturan mimes.



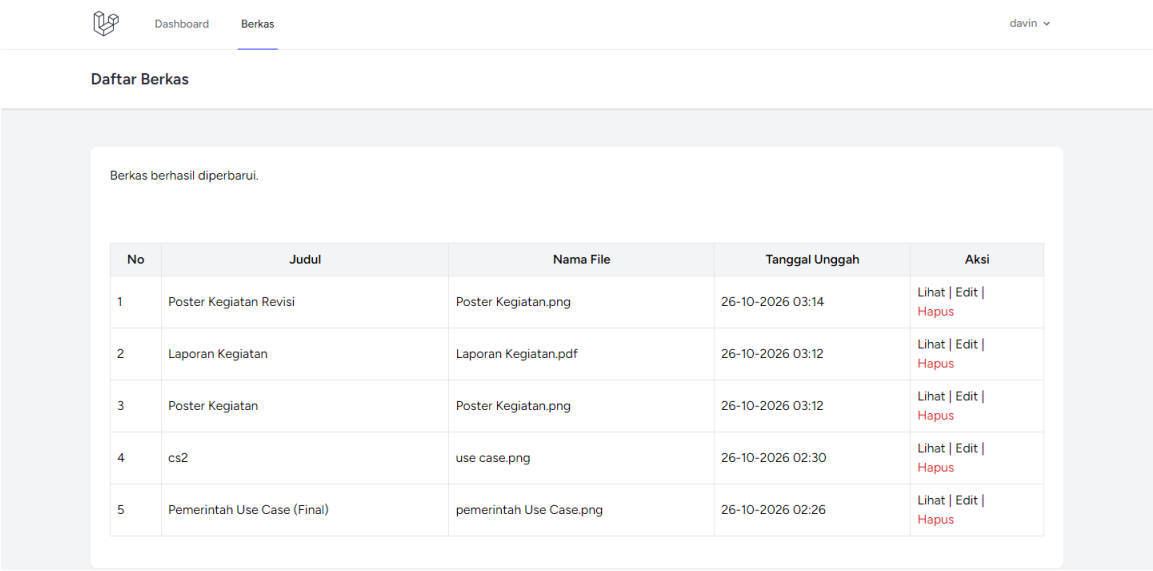
Gambar 11 menampilkan detail berkas gambar. Judul, keterangan, nama file, pemilik, dan tanggal unggah tampil di atas pratinjau gambar.



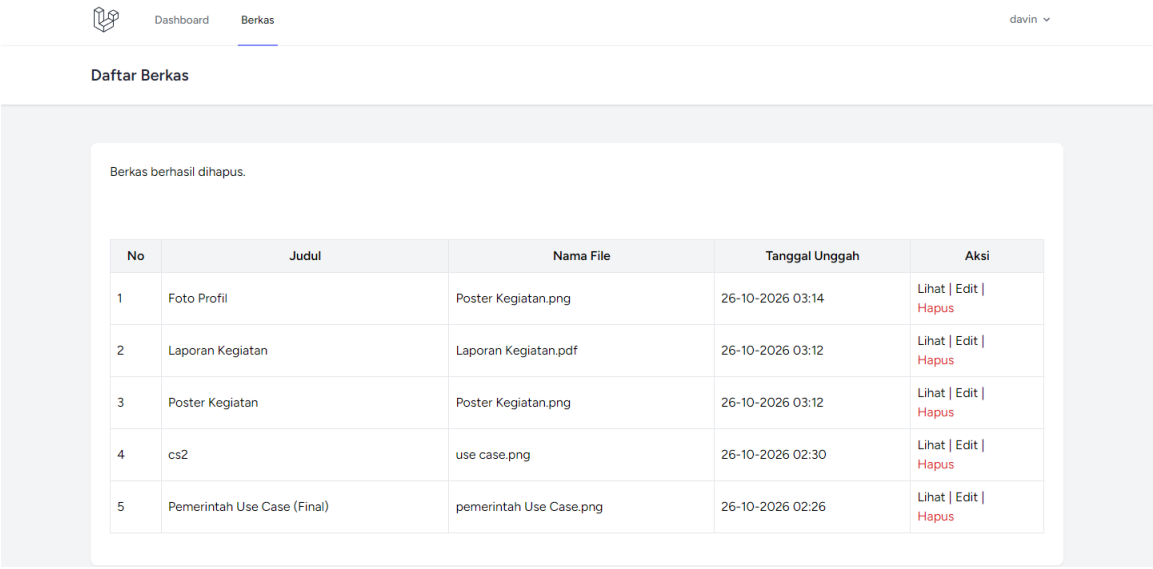
Gambar 12 menampilkan detail berkas PDF. Karena file bukan gambar, halaman menampilkan tombol Buka / Unduh Berkas, bukan pratinjau.



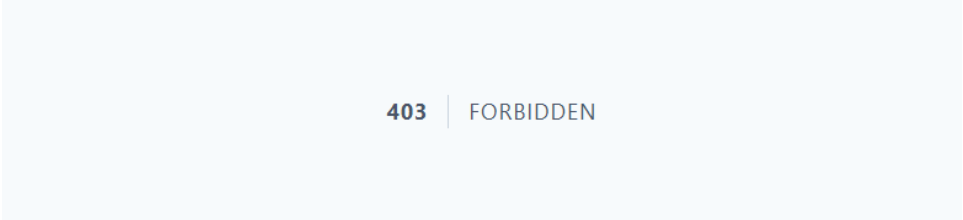
Gambar 13 menampilkan form edit. Judul dan keterangan terisi data lama, tautan File saat ini mengarah ke file yang sedang dipakai, dan input file dibiarkan kosong.



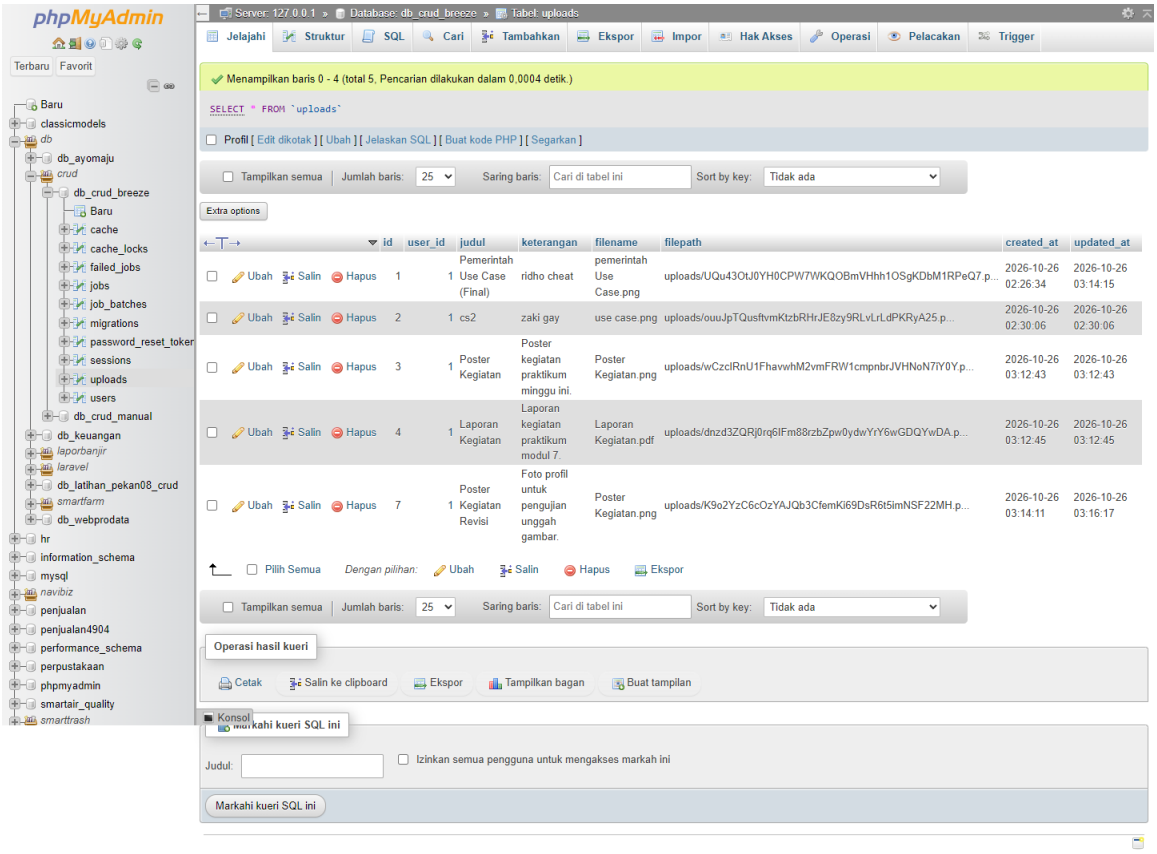
Gambar 14 menampilkan pesan Berkas berhasil diperbarui. Judul Poster Kegiatan berubah menjadi Poster Kegiatan Revisi dan file lamanya tetap dipakai.



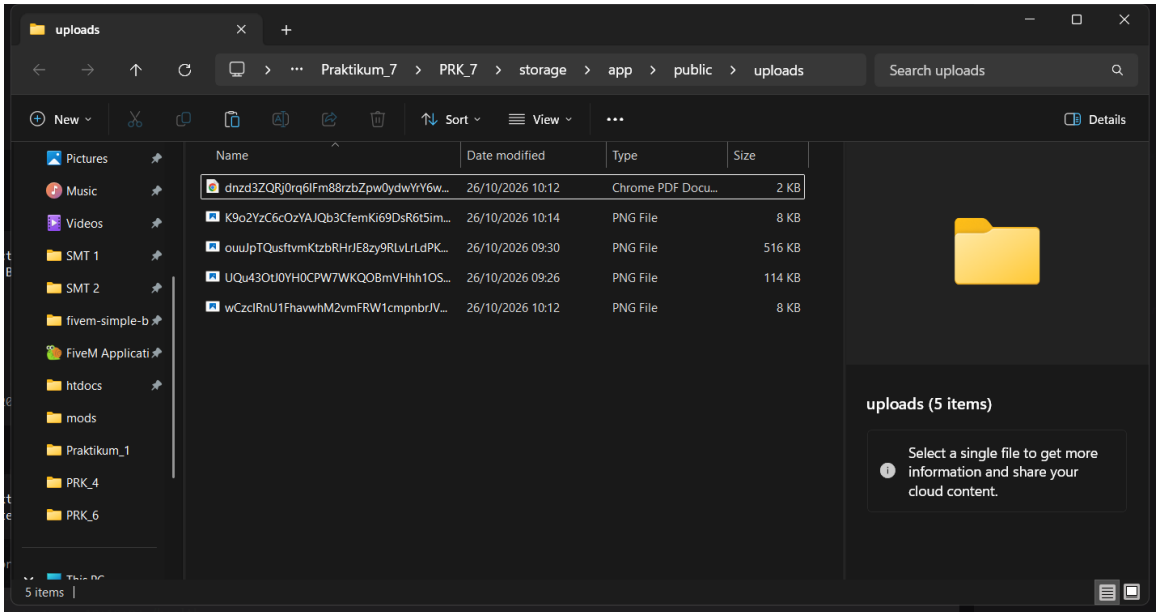
Gambar 15 menampilkan pesan Berkas berhasil dihapus. Setelah satu berkas dihapus, tersisa lima berkas dan pagination tidak lagi muncul.



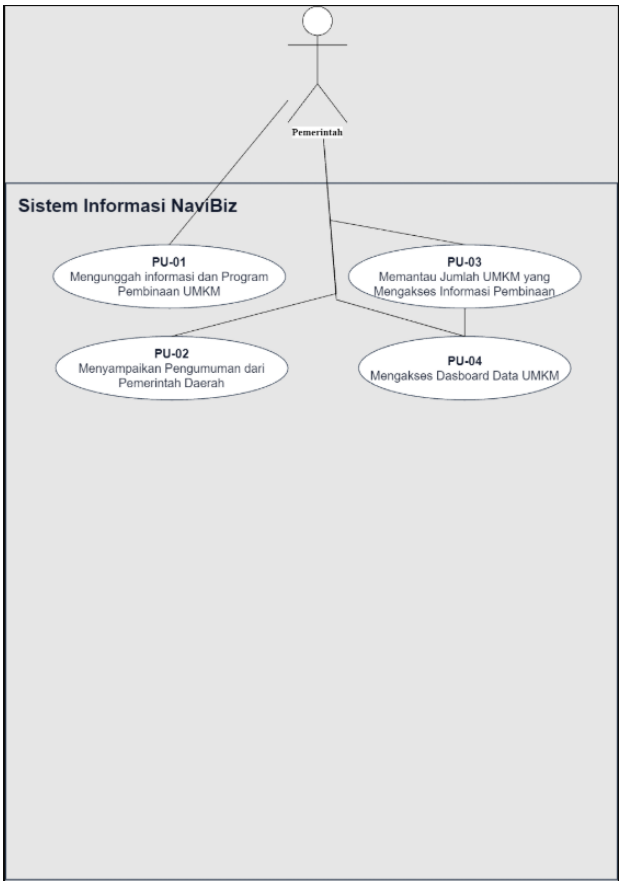
Gambar 16 menampilkan halaman 403 Forbidden ketika akun Pengguna Lain membuka /uploads/1. Pemeriksaan abort_if menolak akses karena user_id berkas bukan milik akun itu.



Gambar 17 menampilkan tabel uploads di phpMyAdmin. Ada lima baris dengan kolom user_id, judul, keterangan, filename, dan filepath. Kolom filepath berisi alamat file di storage, bukan isi file.



Gambar 18 menampilkan isi folder storage/app/public/uploads. Ada lima file dengan nama acak, jumlahnya sama dengan baris di tabel uploads.



Gambar 19 menampilkan file gambar yang dibuka langsung melalui URL storage. File bisa diakses karena symbolic link public/storage sudah dibuat.

5. Analisis dan Pembahasan

1. Pemetaan CRUD ke route dan method controller.

Route::resource('uploads', UploadController::class) memetakan tujuh route ke tujuh method. Halaman daftar memakai uploads.index dan method index(), form tambah memakai uploads.create dan create(), penyimpanan memakai uploads.store dan store(), detail memakai uploads.show dan show(), form ubah memakai uploads.edit dan edit(), penyimpanan perubahan memakai uploads.update dan update(), sedangkan penghapusan memakai uploads.destroy dan destroy(). Operasi update dan delete tidak bisa dikirim langsung dari form HTML karena form hanya mendukung GET dan POST. Karena itu form memakai @method('PUT') dan @method('DELETE') untuk mengubah method request. Route resource membuat pemetaan itu konsisten dan mengurangi penulisan route satu per satu.

2. Penyimpanan file dan path di database.

File disimpan di storage/app/public/uploads dengan nama acak dari method store(). Database hanya menyimpan path file di kolom filepath dan nama asli di kolom filename. Pemisahan itu membuat ukuran database tetap kecil dan file bisa diganti tanpa mengubah struktur tabel. Supaya file bisa dibuka dari browser, folder public/storage harus mengarah ke storage/app/public. Pada project ini tautan itu belum ada saat pertama kali diperiksa, jadi saya menjalankan php artisan storage:link. Setelah tautan dibuat, URL asset('storage/' . $upload->filepath) bisa dipakai untuk pratinjau gambar dan tombol unduh PDF.

3. Validasi dan pembatasan akses.

Validasi di method store dan update memeriksa judul, keterangan, dan file. Aturan mimes membatasi jenis file, sedangkan max:2048 membatasi ukuran maksimal 2 MB. Pesan galat yang muncul masih berbahasa Inggris karena locale aplikasi memakai bawaan Laravel. Pesan itu ditampilkan di bawah input file melalui komponen x-input-error. Selain validasi, setiap method yang menerima parameter Upload memanggil abort_if($upload->user_id !== Auth::id(), 403). Pemeriksaan itu penting karena route model binding hanya mencari data berdasarkan id, tanpa memeriksa pemilik. Tanpa abort_if, pengguna yang sudah login bisa membuka berkas milik pengguna lain hanya dengan menebak id.

4. Pagination dan filter pemilik.

Method index memakai where('user_id', Auth::id())->latest()->paginate(5). Filter user_id membuat daftar hanya berisi berkas milik pengguna yang sedang login. latest() mengurutkan data dari yang terbaru, sedangkan paginate(5) membatasi lima baris per halaman. Pada pengujian, enam berkas menghasilkan dua halaman. Halaman pertama berisi nomor 1 sampai 5, halaman kedua berisi nomor 6. Nomor itu tetap berlanjut karena view memakai $uploads->firstItem() + $loop->index. Setelah satu berkas dihapus, tersisa lima berkas dan pagination tidak lagi muncul.

5. Kendala dan penyelesaian.

Kendala pertama muncul saat file DOCX diunggah. Aturan validasi sudah memuat docx, tetapi Laravel tetap menolak file dengan pesan The file field must be a file of type: jpg, jpeg, png, pdf, doc, docx. Saya memeriksa jenis MIME file DOCX di mesin ini dan hasilnya application/octet-stream, bukan application/vnd.openxmlformats-officedocument.wordprocessingml.document. Deteksi MIME itulah yang membuat aturan mimes gagal. File PDF, JPG, dan PNG tidak mengalami masalah. Kalau fitur DOCX ingin dipakai, aturan validasi bisa ditambah extensions:docx atau deteksi MIME-nya ditangani secara khusus. Saya tidak mengubah kode pada praktikum ini, jadi file DOCX masih ditolak.

Kendala kedua adalah folder public/storage yang belum ada. Tautan storage saya buat dengan php artisan storage:link, lalu pratinjau gambar dan tombol unduh PDF berjalan. Kendala ketiga adalah tautan nama file pada view index memakai kelas text-blue-600underline tanpa spasi. Warna biru dan garis bawah tidak muncul, tetapi tautan tetap bisa diklik. Kendala keempat adalah menu Berkas yang baru ada di navigasi desktop. Menu mobile belum memuat tautan Berkas. Kendala kelima adalah nama kelas model yang tertulis upload dengan huruf kecil. PHP masih menjalankannya, tetapi penulisan itu belum mengikuti PSR-4.

Selama pengujian, tidak ada galat baru yang tercatat di storage/logs/laravel.log. Pesan yang muncul di layar berasal dari validasi yang memang dirancang, yaitu penolakan ZIP, file lebih dari 2 MB, dan DOCX. Saya juga tidak menambahkan feature test khusus uploads. Pengujian CRUD dilakukan manual melalui browser, sedangkan feature test bawaan Breeze tidak mencakup fitur upload.

6. Perbedaan dengan modul.

Ada tiga hal yang berbeda dari modul. Modul mencantumkan docx pada aturan mimes dan menampilkan contoh unggah dokumen, tetapi file DOCX di mesin ini ditolak karena deteksi MIME. Modul meminta menu Berkas ditambahkan pada navigasi desktop dan mobile, sedangkan project ini baru menambahkan menu desktop. Modul juga menampilkan hasil pengujian otomatis Pest pada Gambar 7.33, sedangkan pengujian di laporan ini dilakukan manual melalui browser. Perbedaan itu tidak mengubah alur utama CRUD, tetapi perlu dicatat supaya hasil pengujian tidak dilebih-lebihkan.

6. Kesimpulan

Fitur CRUD dan upload file berhasil dibuat pada project PRK_7. Tabel uploads menyimpan user_id, judul, keterangan, filename, dan filepath. File fisik disimpan di storage/app/public/uploads, dan public/storage dibuat dengan php artisan storage:link. UploadController menjalankan tujuh method resource, route uploads berada di dalam middleware auth, dan view index, create, edit, serta show sudah terhubung.

Pengujian menunjukkan unggah gambar dan PDF berhasil, daftar berkas tampil dengan pagination, detail gambar menampilkan pratinjau, detail PDF menampilkan tombol unduh, judul bisa diubah, dan berkas bisa dihapus. Validasi menolak ZIP dan file lebih dari 2 MB, sedangkan abort_if menolak akses ke berkas milik pengguna lain. Ada dua catatan yang perlu diperbaiki.

File DOCX ditolak oleh validasi mimes karena deteksi MIME di mesin ini, dan menu Berkas baru ada di navigasi desktop. Kedua hal itu tidak menghalangi alur utama CRUD, tetapi perlu ditangani kalau fitur DOCX dan menu mobile ingin dipakai.

7. Link GitHub

Kode program lengkap praktikum ini dapat diakses pada repositori berikut.

https://github.com/davinaileenskie1-hue/PABW-Semester-3